CACODEV

Nonprofit Organization Management System

PROGRAMMER MANUAL

Document ID	RS – 9. PM
Version	1.0
Prepared By	David Katembo, Fabrice Kadima, Pemphyle Nzuzi
Date	2026-03-27
Classification	Internal
References	RS-6, RS-7, RS-8

Table of Contents

Table of Contents 2
Audience 4
3.1 Authentication Component 4
3.2 Member Management Component 4
3.3 Event Management Component 4
3.4 Contribution & Donation Component 5
3.5 Payment Processing Component 5
3.6 AI Assistant Component 5
3.7 Dashboard Component 5
4. Tool Overview 5
4.1 Angular 5
4.2. Java (Spring Boot) 6
4.3. Database-Microsoft SQL Server 6
4.4. Stripe API 6
4.5 OpenAI AI Assistant Integration 7
4.6a Development Tools 7
46.b Testing Tools 7
4.6c Infrastructure & CI/CD Tools 8
5a. Software 8
5b. Test Cases 8
5c. Documentation 9
5d. Test Platform Description 9
5e. Test Scripts 9
Prerequisites 10
Step 1 — Clone the Repository 10
Step 2 — Configure the Backend 10
Step 3 — Build and Run the Backend 11
Step 4 — Install and Run the Frontend 11
Step 5 — Verify Installation 11
Step 1 — Back Up the Database 11
Step 2 - Export Environment Configuration 12
Step 3 - Set Up the Target Environment 12
Step 4 - Restore the Database 12

Step 5 - Update Configuration.....12
Step 6 — Build, Run, and Verify.....12

1. Vision Statement

The Vision of the CACODEV Management System is to give nonprofits a consolidated, scalable, and safe digital platform. The goal of this system is to replace disjointed manual processes like spreadsheets, unofficial routes of communication, and manual payment tracking with a single web-based solution that enhances member involvement, operational effectiveness, and financial transparency.

By giving administrators and members of nonprofit organizations real-time access to organizational data, improved workflows, and simplified internal processes, as well as intelligent support driven by integrated AI technologies, the system seeks to empower them. With a single, unified platform, CACODEV centralizes the administration of donations, events, registrations, and contributions. As a result, the organization is more effective and may grow its influence without adding to its administrative workload.

2. Introduction

This Programmer Manual provides comprehensive technical guidance for developers working on the CACODEV Management System. It describes the system architecture, components, tools, repository organization, installation procedures, and future development opportunities.

The system is a full-stack web application composed of an Angular frontend and a Java Spring Boot backend, with a relational database and external integrations such as Stripe for payment processing and AI services for chatbot functionality.

Audience

- Backend developers working on the Spring Boot API layer
- Frontend developers building or extending Angular components
- QA engineers writing or maintaining test cases
- DevOps engineers managing CI/CD pipelines and deployments
- Project managers requiring a technical overview

3. Component Overview

The system is composed of several core components, each responsible for a specific functional domain.

3.1 Authentication Component

Handles user authentication, authorization, and session management using JWT tokens and Spring Security.
Responsibilities:

- User login and logout
- Token generation and validation
- Role-based access control (RBAC)

3.2 Member Management Component

- Manages member data, and deletes member profiles
- Assign membership types
- Track active/inactive status

3.3 Event Management Component

Supports creation, scheduling, and tracking events and meetings.
Responsibilities:

- Event creation and updates
- Participant registration
- Attendance tracking

3.4 Contribution & Donation Component

Handles financial contributions and donations
Responsibilities:

- Record contributions and donations
- Maintain financial history
- Generate reports

3.5 Payment Processing Component

Integrates with Stipe to process payments securely.
Responsibilities:

- Create Stripe checkout sessions
- Handle webhook events
- Track payment status

3.6 AI Assistant Component

Provides chatbot functionality using OpenAI integration.
Responsibilities:

- Process user queries
- Provide organizational information
- Assist users through conversational interface

3.7 Dashboard Component

Displays analytics and key organizational metrics.
Responsibilities:

- Show real-time statistics
- Display recent activities
- Provides summaries for administrators

4. Tool Overview

The following tools/languages were used in the construction of CACODEV

4.1 Angular

Description:
Frontend framework used to build a responsive single-page application (SPA) built with Angular 19, Angular Material 19, and Bootstrap 5. It communicates exclusively with the backend via REST API calls, using Angular's HttpClient module and RxJS 7 for reactive data streams.

Service	Responsibility
AuthService	JWT generation, validation, and refresh. BCrypt password hashing.
MemberService	Member CRUD, status transitions, cascade-delete safety.
EventService	Event scheduling, organiser assignment, participant coordination.
ContributionService	Approval/rejection workflows, status machine enforcement.
PaymentService	Stripe session creation, webhook verification, subscription management.
AIAssistantService	Proxies requests to OpenAI GPT-4o-mini via Spring AI, handles errors.

Server-Side Rendering (SSR) is enabled via Angular Universal to improve initial page load performance and search engine discoverability.

4.2. Java (Spring Boot)

Description and Usage:
Backend is implemented as a RESTful API using Spring Boot 3.4.4 on Java 17, structured around a Controller–Service–Repository architecture to separate concerns and organize business logic effectively. It exposes all functionality through versioned endpoints under /cacodev/api/*, with Spring Security enforcing role-based access control (RBAC) at the endpoint level. Data persistence and database interactions are handled using Spring Data JPA with Hibernate as the ORM framework, ensuring efficient and consistent data management.

Sub-Component	Responsibility
Auth Module	Login, JWT token storage, route guards, and token refresh logic.
Member Module	Registration wizard, profile management, member list with search and filter.
Event Module	Event calendar, creation forms, organizer assignment, and participant list.
Contribution Module	Submission forms, approval/rejection workflows, and status transitions.
Donation Module	One-time and recurring donation forms, Stripe checkout redirection, and tax receipt download.
AI Assistant	Chat interface backed by OpenAI GPT-4o-mini, with streaming response rendering.
Dashboard	Real-time KPI widgets for admins; self-service summary for members.

4.3. Database-Microsoft SQL Server

All persistent data is stored in Microsoft SQL Server. Hibernate manages the schema through JPA entity mappings. A dedicated test schema is used during all test cycles to isolate test data from production.

4.4. Stripe API

Description and Usage:

Stripe is integrated as an external payment gateway to handle both one-time transactions and recurring billing, with the backend responsible for creating Checkout Sessions, processing webhook events for payment confirmation, and managing the full subscription lifecycle. All interactions are conducted using Stripe test-mode keys exclusively, in compliance with RS-6 Constraint 1, ensuring safe and controlled testing of payment processing, subscription management, and event handling.

4.5 OpenAI AI Assistant Integration

The AI Assistant feature leverages Spring AI to route requests to the OpenAI GPT-4o-mini model, with an integration layer responsible for API key management, request formatting, and error handling. It supports chatbot functionality by answering user queries and generating intelligent responses, while accounting for the model's non-deterministic output by limiting test assertions to HTTP status codes and response structure

4.6 Tool Overview

The CACODEV system relies on a carefully chosen set of tools across development, testing, CI/CD, and operations. Each is described below with its purpose and scope of use.

4.6a Development Tools

Tool / Language	Category	Purpose
Java 17 (LTS)	Language	Primary language for the entire backend layer.
TypeScript 5	Language	Primary language for the Angular frontend.
Spring Boot 3.4.4	Framework	Web, security, data, and AI abstractions for the backend.
Angular 19	Framework	SPA framework with routing, forms, and HTTP client.
Apache Maven 3.9+	Build	Dependency management and build lifecycle for the backend.
Angular CLI 21.2.3	Build	Scaffolding, building, and serving the Angular application.
IntelliJ IDEA / VS Code	IDE	Recommended IDEs for backend and frontend development respectively.

46.b Testing Tools

Tool	Type	Scope
JUnit 5	Unit/Integration	Backend service layer unit and integration tests.
Mockito	Mocking	Mocking dependencies in backend unit tests.
Spring Boot Test	Integration	Full Spring context loading for integration-level tests.
Postman / REST Assured	API	Manual and automated API endpoint validation.
Jasmine + Karma	Frontend Unit	Angular component unit testing.

Tool	Type	Scope
Cypress	E2E	End-to-end browser automation for critical user journeys.
OWASP ZAP	Security	Automated vulnerability scanning (RS-7 Section 8).
SonarQube / SonarLint	Static Analysis	Code quality and security smell detection in CI.
JMeter / k6	Performance	Load and performance benchmarking (deferred to next cycle).

4.6c Infrastructure & CI/CD Tools

- GitHub Actions - CI/CD pipeline executing tests on every push and pull request.
- Stripe CLI - Webhook forwarding for local development and TC-020 testing.
- Microsoft SQL Server - Primary relational database. Test containers recommended for CI isolation.
- Node.js 20+ - Required for Angular CLI and frontend build tooling.

5. Project Repository

The CACODEV repository follows a monorepo structure with clearly separated frontend and backend directories. Below is a guided walkthrough of each key area.

5a. Software

```
cacodev/
├── backend/                  # Spring Boot application (Maven)
│   ├── src/main/java/        # Application source code
│   │   └── com/cacodev/
│   │       ├── auth/         # Authentication & JWT
│   │       ├── member/       # Member management
│   │       ├── event/        # Event management
│   │       ├── contribution/ # Contribution tracking
│   │       ├── donation/     # Donation management
│   │       ├── payment/      # Stripe integration
│   │       ├── ai/           # OpenAI AI assistant
│   │       └── config/       # Security, CORS, Stripe, AI config
│   ├── src/main/resources/
│   │   └── application.yml   # Environment configuration
│   └── pom.xml               # Maven dependencies
└── frontend/                 # Angular 19 SPA
    ├── src/app/
    │   ├── auth/             # Login, guards, interceptors
    │   ├── members/          # Member module
    │   ├── events/           # Event module
    │   ├── contributions/    # Contribution module
    │   ├── donations/        # Donation module
    │   ├── ai/               # AI chat interface
    │   └── dashboard/        # Dashboard module
    ├── angular.json
    └── package.json
```

5b. Test Cases

Test cases are defined in document RS-7. The corresponding automated test files live in the repository as follows:

```
backend/src/test/java/com/cacodev/
├── auth/           # TC-001 to TC-004 (JUnit 5 + Spring Boot Test)
├── member/         # TC-005 to TC-008
├── event/          # TC-011 to TC-013
├── contribution/   # TC-014 to TC-016
├── payment/        # TC-017 to TC-020
└── security/       # TC-021 to TC-023

frontend/cypress/e2e/
├── auth.cy.ts      # TC-001, TC-002
├── members.cy.ts   # TC-005 to TC-008
├── events.cy.ts    # TC-011, TC-012
├── donations.cy.ts # TC-017, TC-018
└── ai.cy.ts        # TC-024
```

5c. Documentation

All project documentation is maintained in the docs/ directory at the repository root:

- RS-6 – Software Test Plan
- RS-7 – Test Cases
- RS-8 – Test Summary Report
- PM-1 – This Programmer Manual
- API documentation is auto generated via SpringDoc OpenAPI at /swagger-ui/index.html when the backend is running.

5d. Test Platform Description

Component	Details
Operating System	Windows 11 (developer) / Linux Ubuntu (CI pipeline)
JDK	Java 17 LTS
Node.js	Node.js 20+
Frontend Dev Server	Angular CLI — http://localhost:4200
Backend API Server	Spring Boot (embedded Tomcat) — http://localhost:8080
API Base Path	http://localhost:8080/cacodev/api/*
API Documentation	http://localhost:8080/swagger-ui/index.html
Database	Microsoft SQL Server (dedicated test schema)
Browser Compatibility	Chrome (P1), Firefox (P1), Edge (P2), Safari (P2) — all latest stable

5e. Test Scripts

The following commands run the full test suite from the repository root:

```
# Backend unit and integration tests
cd backend && mvn test

# Backend tests with coverage report
cd backend && mvn verify jacoco:report

# Frontend unit tests (Jasmine + Karma)
cd frontend && npm run test

# Frontend unit tests (headless CI mode)
cd frontend && npm run test -- --watch=false --browsers=ChromeHeadless

# Cypress E2E tests (interactive)
cd frontend && npx cypress open

# Cypress E2E tests (headless CI mode)
cd frontend && npx cypress run

# OWASP ZAP security scan (requires ZAP running on port 8090)
zap-cli quick-scan --self-contained --start-options '-config api.disablekey=true'
http://localhost:4200
```

6. Installation — New Install

Follow these steps to install CACODEV on a clean environment with no prior data.

Prerequisites

- Java 17 LTS installed and JAVA_HOME configured
- Apache Maven 3.9+ on PATH
- Node.js 20+ and npm installed
- Microsoft SQL Server running and accessible
- A Stripe account (test mode) with API keys
- An OpenAI API key with access to GPT-4o-mini
- Git installed

Step 1 — Clone the Repository

```
git clone https://github.com/cacodev/cacodev.git
cd cacodev
```

Step 2 — Configure the Backend

Create or edit backend/src/main/resources/application.yml with your environment values:

```
spring:
  datasource:
    url: jdbc:sqlserver://localhost:1433;databaseName=cacodev_db
    username: YOUR_DB_USERNAME
    password: YOUR_DB_PASSWORD
```

```
  jpa:
    hibernate:
      ddl-auto: update   # Use 'create' on first run to generate schema

stripe:
  secret-key: sk_test_YOUR_STRIPE_SECRET_KEY
  webhook-secret: whsec_YOUR_WEBHOOK_SECRET

openai:
  api-key: YOUR_OPENAI_API_KEY

app:
  jwt-secret: YOUR_STRONG_JWT_SECRET
  jwt-expiration-ms: 86400000
```

Step 3 — Build and Run the Backend

```
cd backend
mvn clean install -DskipTests
mvn spring-boot:run
# Backend available at: http://localhost:8080
# Swagger UI at: http://localhost:8080/swagger-ui/index.html
```

Step 4 — Install and Run the Frontend

```
cd cacodev-ui
npm install
ng serve
# Frontend available at: http://localhost:4200
```

Step 5 — Verify Installation

1. Open http://localhost:4200 in Chrome or Firefox.
2. Log in with the default admin credentials (set in your data seed or first-run script).
3. Navigate to the Dashboard - confirm member count, event count, and contribution widgets load.
4. Visit the Swagger UI and execute a GET /api/members request - expect HTTP 200.
5. Send a test message in the AI Assistant - expect a response with HTTP 200.

7. Installation — New Platform (Data Preserved)

Use this procedure when migrating CACODEV to a new server or environment while preserving all existing data. This process assumes you have access to the source environment.

Step 1 — Back Up the Database

On the source SQL Server, perform a full database backup:

```
-- Run in SQL Server Management Studio or sqlcmd
BACKUP DATABASE cacodev_db
TO DISK = 'C:\Backups\cacodev_db_backup.bak'
WITH FORMAT, COMPRESSION, STATS = 10;
```

Step 2 - Export Environment Configuration

Copy the following files from the source environment to a secure location:

- backend/src/main/resources/application.yml (or equivalent environment variables)
- Any SSL/TLS certificates in use
- Stripe webhook signing secrets

Step 3 - Set Up the Target Environment

On the new platform, install all prerequisites (see Section 6, Prerequisites) and clone the repository at the same Git tag or commit as the source environment.

Step 4 - Restore the Database

```
-- On target SQL Server
RESTORE DATABASE cacodev_db
FROM DISK = 'C:\Backups\cacodev_db_backup.bak'
WITH MOVE 'cacodev_db' TO 'C:\Data\cacodev_db.mdf',
     MOVE 'cacodev_db_log' TO 'C:\Data\cacodev_db_log.ldf',
     REPLACE, STATS = 10;
```

Step 5 - Update Configuration

Edit application.yml on the target environment with the new database host/credentials. Set the Hibernate DDL strategy to validate (not update or create) to protect restored data:

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate   # CRITICAL: prevents unintended schema changes
```

Step 6 — Build, Run, and Verify

Follow Steps 3 and 4 from Section 6 to build and start the backend and frontend. Verify that:

- All existing members, events, and contributions are visible in the dashboard.
- Member login works with pre-existing credentials.
- No HTTP 500 errors appear in the backend logs on first startup.
- The schema validation passes (check backend logs for 'Schema-validation: missing table' errors).

8. Further Development Statement

With an additional year of development, CACODEV could be enhanced through several high-impact improvements:

- Multi-tenancy and SaaS transformation: Refactor the platform into a multi-tenant architecture to allow multiple nonprofit organizations to onboard quickly, with strict data isolation and shared infrastructure.

- Native mobile applications: Develop iOS and Android apps using React Native or Flutter, leveraging the existing REST API to enable push notifications and mobile administrative actions.
- Advanced AI capabilities: Extend the AI assistant with context-aware features such as automated event summaries, financial trend analysis, natural language data queries, and anomaly detection in contribution patterns.
- Automated regulatory compliance reporting: Implement a document generation system to produce financial summaries, tax documents, and required regulatory filings based on system data.
- Performance testing and scalability: Complete performance testing, then containerize the system with Docker and deploy on Kubernetes to support horizontal scaling and zero-downtime updates.
- Expanded test coverage: Increase backend test coverage to at least 80% for service-layer components, introduce Test containers for better isolation, and complete previously omitted test scenarios.
- Enhanced access control: Implement fine-grained role-based access control with more detailed permission management.
- Advanced analytics and dashboards: Expand reporting capabilities with richer data visualization and insights.
- Payment provider integration: Add support for additional payment gateways beyond the current implementation.
- Internationalization: Introduce multi-language support to broaden accessibility.
- Microservices evolution: Improve performance and maintainability by transitioning toward a microservices-based architecture.
- Automated DevOps pipelines: Establish CI/CD pipelines to streamline testing, deployment, and system operations.